

KIX

Uma Especificação Estruturada para
Infraestruturas de Pagamento Lightning

WHITE PAPER

Especificação Técnica

Autor: Hexarcs

KIX Protocol · Arquitetura P.R.I.S.M.A.

Visão Geral da Arquitetura KIX

Camadas P.R.I.S.M.A., da interface POS / carteira até a infraestrutura de hospedagem.

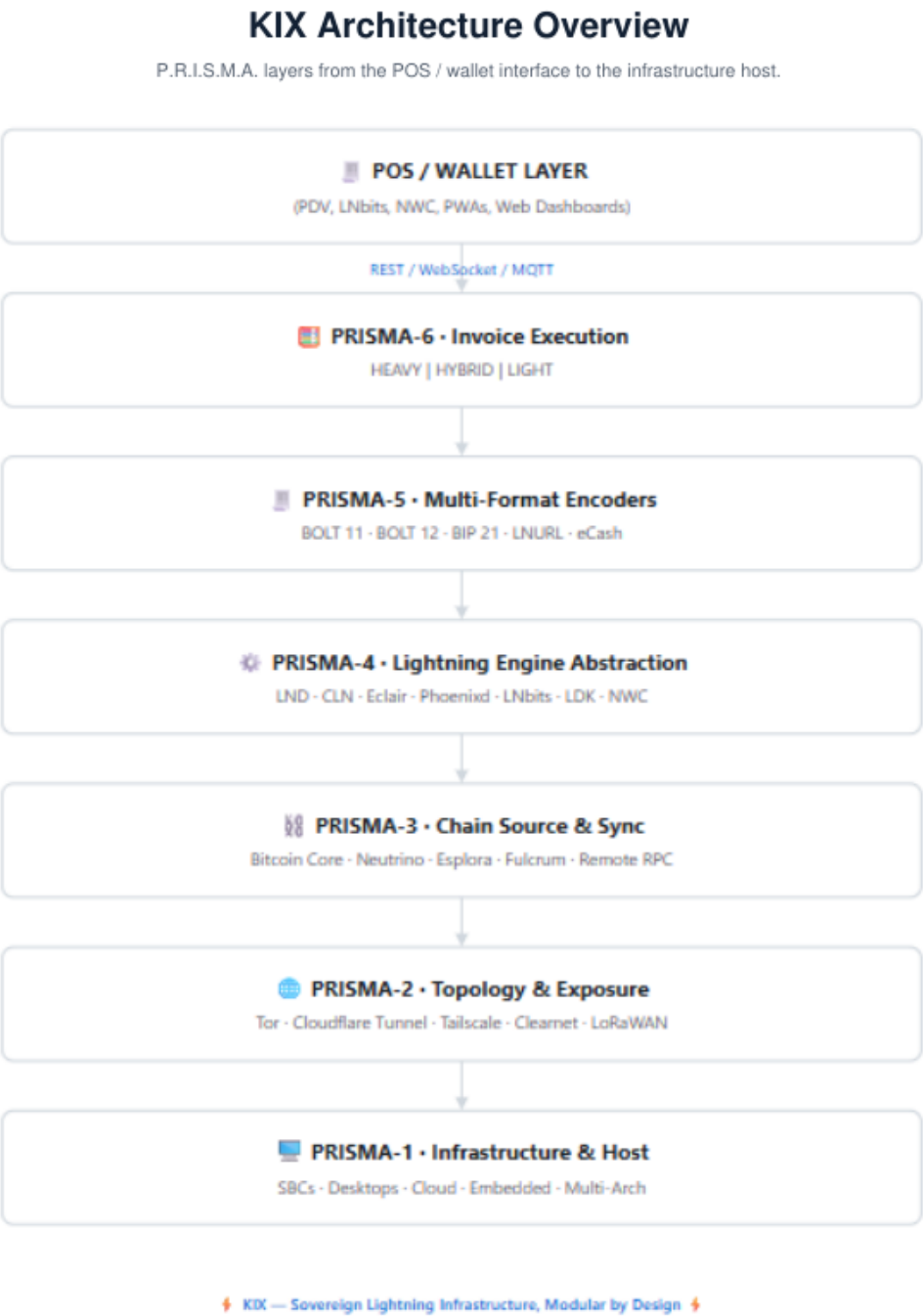


Figure 1. Layered view of the KIX payment infrastructure and its six PRISMA modules.

Figura 1. Visão em camadas da infraestrutura de pagamentos KIX e de seus seis módulos P.R.I.S.M.A.

Resumo

A complexidade de implementação e comunicação nas redes de pagamento de segunda camada frequentemente limita sua adoção em larga escala. Propomos o KIX, uma especificação de protocolo de código aberto projetada para reduzir a carga cognitiva, fragmentando a arquitetura da Lightning Network em estágios modulares e padronizados.

Em vez de exigir que cada operador descreva exaustivamente sua própria pilha de hardware e software para interagir com o ecossistema, o protocolo padroniza a comunicação técnica, permitindo que a comunidade se refira a camadas arquiteturais universais. O modelo, fundamentado no KIX Core, concentra-se estritamente na implementação prática da rede, oferecendo aos usuários a flexibilidade de abstrair a complexidade técnica subjacente, caso desejem.

Como uma especificação baseada em elementos de código aberto, o KIX alinha os incentivos estruturais necessários para escalar uma infraestrutura de pagamentos de maneira soberana, descentralizada e não custodial.

1. Introdução

A operação soberana de um nó Bitcoin completo, juntamente com implementações tradicionais da Lightning Network — como CLN, LND ou Eclair — frequentemente representa uma barreira técnica intimidadora para comerciantes locais e indivíduos comuns. Ao longo dos anos, a comunidade concentrou esforços no desenvolvimento de diversas tecnologias e abstrações para reduzir essa fricção inicial, com o objetivo constante de mitigar a complexidade e diminuir drasticamente os requisitos computacionais necessários para utilizar a rede de pagamentos no dia a dia.

Apesar desses avanços estruturais, observa-se que até mesmo desenvolvedores de software e engenheiros imersos no ecossistema Lightning muitas vezes desconhecem a ampla diversidade de hardware que pode ser empregada na implementação prática da rede. A padronização arquitetural proposta pelo protocolo KIX facilita ativamente o desenvolvimento do ecossistema ao criar uma linguagem técnica comum, permitindo que a comunidade compreenda capacidades operacionais que vão de servidores corporativos em nuvem a microcontroladores extremamente baratos e de baixo consumo.

Além disso, a especificação desempenha um papel importante na prevenção de uma “visão de túnel” entre os implementadores em relação às topologias de comunicação. O KIX explicita que diferentes abordagens de exposição e rede podem ser adotadas — inclusive executadas em paralelo — definindo precisamente em qual estágio da arquitetura de um nó o sistema pode ser escalado horizontalmente utilizando ferramentas de containerização, como Docker e seus derivados.

Por fim, a organização do protocolo posiciona arquiteturalmente o ecossistema de software de pagamentos. Ela determina exatamente onde se situam soluções de ponto de venda (POS) e ecossistemas de gerenciamento de carteiras — como LNbits ou interfaces via NWC (Nostr Wallet Connect). Em essência, a especificação mapeia como módulos de nível superior se comunicam por meio de requisições de API com os mecanismos inferiores da pilha, garantindo que o processamento pesado e o roteamento de invoices sejam abstraídos e delegados às camadas apropriadas, liberando o usuário final da complexidade criptográfica e computacional subjacente.

2. PRISMA-1: Execução da Infraestrutura e Perfis de Host

PRISMA-1 estabelece as bases físicas e operacionais do protocolo KIX, definindo diretrizes e especificações para ambientes de hardware e sistemas operacionais (com suporte a ambientes Linux, Android e Windows) capazes de sustentar uma implementação de acordo com a arquitetura P.R.I.S.M.A. A escolha do sistema operacional e da plataforma de execução impacta profundamente o comportamento dos demais módulos do protocolo, definindo desde a disponibilidade de mecanismos nativos de roteamento Lightning até estratégias de exposição de rede e armazenamento. O objetivo fundamental deste módulo é fornecer um modelo de referência soberano para a camada de

execução.

2.1 Arquiteturas e Requisitos de Instruções

O modelo orienta o uso de ambientes de 64 bits para garantir a interoperabilidade das toolchains. Embora a filosofia seja multiarquitetura (x86_64, ARM64 e, futuramente, RISC-V), o padrão de 64 bits é fundamental para manter a estabilidade dos componentes críticos.

2.2 Dimensionamento e Impacto sobre Recursos

- Perfis Restritos / Embarcados (ARM64 / SBCs): dispositivos como NanoPi ou Orange Pi operam sob restrições severas (1 GB a 4 GB de RAM), exigindo estratégias leves de sincronização.
- Perfis Expandidos (x86_64 / Servidores e Nuvem): ambientes locais ou em nuvem com maior capacidade, permitindo a execução simultânea de mecanismos Lightning completos, indexadores e back-office.

2.3 Composição Modular por S.H.A.R.D.s (Software Hardware Automated Routing Division)

Uma Infrastructure S.H.A.R.D. representa a unidade física e operacional básica do ambiente de hospedagem. Exemplos:

- Micro-Embarcado: terminal Android (Sunmi V2) ou SBC ultracompacto. Focado em chamadas de API por conexão remota.
- Edge / Chão de Loja: SBC (NanoPi, Raspberry Pi 4) para servidor local de balcão.
- Desktop Autônomo: PC/Mini-PC dedicado para executar um nó local completo e microsserviços POS.
- Nó em Nuvem / Enterprise: instância virtualizada atuando como servidor centralizador (Cluster/Multi-Tenant).

3. PRISMA-2: Topologia, Deploy e Exposição de Rede

PRISMA-2 define as diretrizes para containerização, isolamento de processos e estratégias de exposição de rede. O objetivo é padronizar como os microsserviços KIX são empacotados, orquestrados e disponibilizados, abrangendo desde ambientes de containerização (Docker, Podman) até execução direta (bare-metal).

3.1 Composição Modular por S.H.A.R.D.s de Deploy e Exposição

- Darknet Soberana: endereços Tor .onion v3 isolados em containers, com foco em soberania e privacidade totais.
- Túnel Seguro: orquestração dinâmica expondo serviços via Cloudflare Tunnel ou redes VPN mesh (Tailscale), com baixa latência.
- Clearnet Pública: execução nativa ou em host-network, exposta diretamente via IPv4/IPv6 com reverse proxy (Nginx/Caddy) e TLS automático.
- Mesh e Resiliência Offline: transmissão via rádio LoRaWAN ou redes p2p locais (Wi-Fi/Bluetooth) para locais sem cobertura de internet.

4. PRISMA-3: Fonte da Chain e Sincronização do Backend

PRISMA-3 especifica as diretrizes para conexão, validação de estado e sincronização com a blockchain Bitcoin, garantindo que a aplicação determine o estado real da rede com soberania, fornecendo os dados aos mecanismos de liquidação.

4.1 Mecanismos de Sincronização e Fontes de Dados

- Full Soberano: execução direta do Bitcoin Core (Full/Pruned Node) com validação local completa (Zero-Trust), acoplada via ZMQ e RPC.
- Light Compacto: clientes leves via filtros compactos (BIP 157/158), como Neutrino, com extrema eficiência de armazenamento.
- Gateway Indexado: camadas intermediárias (Esplora / Fulcrum) para consultas instantâneas e suporte ao mempool.
- RPC Remoto / Relay de Provedor: middleware conectando o POS a nós remotos administrados por terceiros, com foco em baixa latência e sem sincronização local.

5. PRISMA-4: Abstração do Mecanismo Lightning

PRISMA-4 especifica a camada de abstração e alternância entre mecanismos de liquidação, tornando as interfaces POS completamente agnósticas quanto ao backend de pagamentos. Ela permite alternar dinamicamente entre mecanismos executados localmente ou hospedados por terceiros.

5.1 Composição Modular por S.H.A.R.D.s de Mecanismos Lightning

- Native Enterprise: mecanismos diretos, como LND ou Core Lightning, para nós soberanos de alto volume.
- Modular Engine: LNbits ou Alby Hub local, ideal para ecossistemas multi-tenant e isolamento entre lojas.
- Autonomous Server: mecanismos como Phoenixd, focados no gerenciamento automático de liquidez por meio de splicing.
- Delegated Remote: carteiras remotas via Nostr Wallet Connect (NWC), conectadas como cliente sem infraestrutura de nó local.
- Embedded SDK: provedores de API ou backends em nuvem baseados em LDK (Lightning Development Kit), com as chaves mantidas pelo cliente.

6. PRISMA-5: Codificadores de Invoice em Múltiplos Formatos

PRISMA-5 garante máxima interoperabilidade no POS, gerando e convertendo dinamicamente formatos nativos de cobrança da rede e esquemas proprietários de contingência.

6.1 Formatos Padronizados e Estratégias de Fallback

- Padrão Nativo: geração nativa de invoices BOLT 11 (uso único) e suporte a Offers (BOLT 12).
- Universal Unificado: QR Codes BIP 21 agrupando On-Chain e Lightning, integrados à resolução LNURL-Pay para abranger qualquer carteira do cliente.
- Contingência e Tokenização: conversão para tokens eCash (Cashu/Fedimint) ou vouchers de crédito locais durante interrupções da rede. Emite invoices contingentes com liquidação enfileirada para compensação posterior, sem interromper o caixa.

7. PRISMA-6: Perfis de Execução de Invoice

PRISMA-6 especifica a distribuição da carga computacional entre o terminal de ponto de venda (POS) e a infraestrutura de back-office, desacoplando a lógica do terminal da necessidade de recursos elevados.

7.1 Distribuição da Carga Computacional

- Perfil Heavy (PRISMA-6-HEAVY): servidores (x86_64/ARM64) executando frameworks web (Django, FastAPI), gerenciando bancos de dados, nós Lightning completos e webhooks síncronos de e-commerce.
- Perfil Intermediate (PRISMA-6-HYBRID): dispositivos POS Android portáteis (Termux/Sunmi) capazes de executar PWAs e aplicações nativas, gerenciando certificados locais e impressão de recibos via API de nuvem.
- Perfil Light (PRISMA-6-LIGHT): microcontroladores (ESP32, STM32 com displays TFT). Eles atuam puramente como interface visual. Transmitem o valor do pedido via MQTT, WebSockets ou REST para o servidor HEAVY, exibem o QR Code retornado e aguardam o callback sem processar criptografia localmente.